Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 3:

Polyalphabetic Ciphers: The Vigenère Cipher

Elaborated: st. gr. FAF-243
Șoimu Ionuț

Verified: asist. univ.
Zaica Maia

Chișinău – 2026

Contents

1 Introduction and Objectives **1**

2 Short Theory **2**
2.1 Principles of Polyalphabetic Substitution 2
2.2 The Vigenère Cipher 2
2.3 Mathematical Model over the Romanian Alphabet (n = 31) 2
2.3.1 Encryption Formula 2
2.3.2 Decryption Formula 3
2.4 The Tabula Recta 3

3 Implementation Details **3**
3.1 Architecture and Romanian Alphabet Representation 3
3.2 Code Fragments and Explanations 3
3.2.1 Alphabet Definition and Explicit Mapping 3
3.2.2 Diacritic Normalization and Cedilla Compatibility 4
3.2.3 Whitespace Stripping and Strict Character Validation 4
3.2.4 Key Validation and Length Enforcement (|K| >= 7) 5
3.2.5 Vigenère Encryption and Decryption Core 5
3.2.6 Step-by-Step Mathematical Calculation Table 6

4 Results and Demonstration **7**
4.1 Task 3.2: Operational Verification and Step-by-Step Calculation 7
4.2 Task 3.2: Round-Trip Decryption Verification 8
4.3 Task 3.2: Error Handling and Input Validation 8
4.4 Interactive CLI Application 8

5 Conclusions **9**

6 Source Code Repository **9**

1 Introduction and Objectives

Classical cryptography provides foundational mechanisms for confidential communications. In previous laboratory assignments, monoalphabetic substitution systems were studied and evaluated: the classical Caesar cipher and the keyword-permuted Caesar cipher (Laboratory 1), followed by the cryptanalysis of monoalphabetic ciphers via statistical frequency analysis (Laboratory 2). Because monoalphabetic ciphers apply a fixed, stationary mapping from plaintext characters to ciphertext characters, the natural unigram, bigram, and trigram frequencies of the source language remain completely preserved in the ciphertext, allowing an adversary to decipher intercepted traffic without possessing the secret key.

To overcome this structural vulnerability, classical cryptographic theory developed **polyalphabetic substitution ciphers**. The defining characteristic of a polyalphabetic system is the systematic alternation among multiple distinct substitution alphabets throughout the message. Consequently, a high-frequency plaintext letter (such as `E` or `A`) is mapped to different ciphertext symbols at different positions, while identical ciphertext symbols can originate from distinct plaintext letters. This alternating substitution flattens the frequency distribution of the ciphertext and conceals the underlying linguistic statistics.

In accordance with the guidelines for Laboratory Work No. 3 (*Lucrare de laborator nr. 3. Cifruri polialfabetice*), students with even catalog numbers are assigned **Task 3.2: The Vigenère Cipher for the Romanian Alphabet**.

The primary objectives of this laboratory work are:

1. **Understand the theoretical foundations** and mathematical mechanics of polyalphabetic substitution, specifically how rotating through multiple Caesar shifts flattens language frequency distributions.

2. **Adapt the Vigenère cipher to the 31-letter Romanian alphabet**, establishing an exact bijective mapping onto the commutative ring $\mathbb{Z}_{31} = \{0, 1, \ldots, 30\}$ without relying on language-dependent ASCII or Unicode codepoint shifts.

3. **Implement standard Romanian diacritic handling**, correctly supporting standard comma-below letters (Ș, Ț – Unicode `U+0218`, `U+021A`) while automatically normalizing legacy cedilla characters (Ş, Ţ – Unicode `U+015E`, `U+0162`).

4. **Enforce strict input preprocessing and validation** as mandated by Task 3.2: spaces must first be stripped from the plaintext message prior to uppercase transformation, non-alphabet characters (digits, punctuation, foreign symbols) must be detected and rejected with an explicit diagnostic indicating the allowed range, and keys must satisfy the minimum length threshold $|K| \geq 7$.

5. **Implement algebraic encryption and decryption** using modular arithmetic in $\mathbb{Z}_{31}$, taking advantage of floored division to guarantee non-negative residues.

6. **Generate step-by-step mathematical calculation tables** formatted across 6 rows, reproducing the exact structure demonstrated in Section 3.1 of the laboratory manual.

7. **Construct a clean, modular CLI and interactive menu system** consistent with the layout and design conventions established in Laboratory 1.

2 Short Theory

2.1 Principles of Polyalphabetic Substitution

The fundamental weakness of any monoalphabetic substitution cipher is that its output frequency histogram reflects the probability distribution of the source language. A cipher is cryptographically stronger when its output distribution is uniform, offering no statistical bias to a cryptanalyst.

Polyalphabetic ciphers flatten this distribution by combining high-frequency letters with low-frequency letters. If a common letter (e.g., `T`) is encrypted sometimes as `A` and other times as `B`, and if a rare letter (e.g., `X`) is also encrypted sometimes as `A` and other times as `B`, the high probability of `T` merges with the low probability of `X`, yielding an intermediate, balanced frequency for both `A` and `B`. This alternation is achieved by employing multiple substitution alphabets in cyclic succession. The earliest known polyalphabetic system was described by Leon Battista Alberti in 1568, using a mechanical cipher disk.

2.2 The Vigenère Cipher

The Vigenère cipher was historically misattributed in the 19th century to Blaise de Vigenère; it was actually first published by the Italian cryptologist Giovan Battista Bellaso in his 1553 treatise *La cifra del. Sig. Giovan Battista Bellaso*.

The cipher operates as a sequence of Caesar shifts, where the shift value is not constant but varies cyclically according to a secret key string $K = (k_0, k_1, \ldots, k_{L-1})$. Each character of the key determines a specific additive shift in the range $0 \leq k_j < n$, where n is the alphabet cardinality. The key is repeated periodically across the length of the message:

$$k'_i = k_{i \bmod L}, \quad \text{for } i = 0, 1, \ldots, N-1. \tag{1}$$

2.3 Mathematical Model over the Romanian Alphabet ($n = 31$)

Let Σ represent the canonical Romanian alphabet of $n = 31$ uppercase letters:

$$\Sigma = (\mathrm{A}, \breve{\mathrm{A}}, \hat{\mathrm{A}}, \mathrm{B}, \mathrm{C}, \mathrm{D}, \mathrm{E}, \mathrm{F}, \mathrm{G}, \mathrm{H}, \mathrm{I}, \hat{\mathrm{I}}, \mathrm{J}, \mathrm{K}, \mathrm{L}, \mathrm{M}, \mathrm{N}, \mathrm{O}, \mathrm{P}, \mathrm{Q}, \mathrm{R}, \mathrm{S}, \mathrm{\c{S}}, \mathrm{T}, \mathrm{\c{T}}, \mathrm{U}, \mathrm{V}, \mathrm{W}, \mathrm{X}, \mathrm{Y}, \mathrm{Z}). \tag{2}$$

Table 1 defines the bijective encoding $\mathrm{enc} : \Sigma \to \mathbb{Z}_{31}$ assigning each letter an integer index $x \in \{0, 1, \ldots, 30\}$.

Table 1: Canonical Romanian Alphabet Letter Encoding ($n = 31$, Table 2)

Letter	A	Ă	Â	B	C	D	E	F	G	H	I	Î	J	K	L	M
Code	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Letter	N	O	P	Q	R	S	Ș	T	Ț	U	V	W	X	Y	Z	–
Code	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	–

Let the preprocessed plaintext message be $M = (m_0, m_1, \ldots, m_{N-1})$, where each $m_i \in \mathbb{Z}_{31}$, and let the secret key be $K = (k_0, k_1, \ldots, k_{L-1})$, where $k_j \in \mathbb{Z}_{31}$ and $L = |K| \geq 7$.

2.3.1 Encryption Formula

Each ciphertext character c_i is computed through modular addition:

$$c_i = (m_i + k'_i) \pmod{31}. \tag{3}$$

2.3.2 Decryption Formula

Decryption is the exact inverse operation under modular arithmetic, subtracting the periodic key index:

$$m_i = (c_i - k'_i) \pmod{31}. \tag{4}$$

When the difference $(c_i - k'_i)$ is negative, the value wraps around into the non-negative residue domain $[0, 30]$:

$$(c_i - k'_i) \pmod{31} \equiv ((c_i - k'_i) \bmod 31 + 31) \bmod 31. \tag{5}$$

In Python, the modulo operator `%` implements floored division, which automatically produces a non-negative residue within $[0, 30]$ for any positive modulus.

2.4 The Tabula Recta

The Tabula Recta is a 31×31 square matrix wherein each row represents a cyclic left shift of the Romanian alphabet by one position. The row index corresponds to the key letter k'_i, the column index corresponds to the plaintext letter m_i, and their intersection yields the ciphertext letter c_i. The Tabula Recta provides a visual, geometric representation that is isomorphic to modular addition in $\mathbb{Z}_{31}$.

3 Implementation Details

3.1 Architecture and Romanian Alphabet Representation

The implementation was constructed in Python with a clean, modular architecture separated into distinct components within `lab3/`:

- `romanian_alphabet.py`: Canonical definition of the 31-letter Romanian alphabet, bidirectional mapping dictionaries, diacritic normalizers, text preprocessing routines, and key/message validators.
- `vigenere_cipher.py`: Task 3.2 core encryption and decryption algorithms, step-by-step mathematical matrix formatting, and interactive sub-menu routines.
- `main.py`: Top-level CLI entry point supporting interactive menu navigation, Task 3.2 execution, and command-line argument parsing.

3.2 Code Fragments and Explanations

3.2.1 Alphabet Definition and Explicit Mapping

The assignment strictly requires that built-in ASCII/Unicode codepoint arithmetic must not be used to compute cipher shifts. Instead, an explicit list and bidirectional dictionary mappings are established:

```
ROMANIAN_ALPHABET: List[str] = [
    'A', 'Ă', 'Â', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I',
    'Î', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S',
    'Ș', 'T', 'Ț', 'U', 'V', 'W', 'X', 'Y', 'Z'
]
ALPHABET_SIZE: int = len(ROMANIAN_ALPHABET)  # 31

CHAR_TO_INDEX: Dict[str, int] = {char: idx for idx, char in enumerate(
    ROMANIAN_ALPHABET)}
```

```
INDEX_TO_CHAR: Dict[int, str] = {idx: char for idx, char in enumerate(
    ROMANIAN_ALPHABET)}
```

Comment: This defines the exact domain $\mathbb{Z}_{31}$. Mapping lookup is $O(1)$ and guarantees that letter substitutions strictly follow the canonical Table 2 indices rather than arbitrary ASCII/Unicode offsets.

3.2.2 Diacritic Normalization and Cedilla Compatibility

Romanian typography frequently encounters Turkish-style cedilla variants (Ş, Ţ) instead of standard Romanian comma-below letters (Ș, Ț). An explicit normalizer handles this transparently:

```
CEDILLA_MAPPING: Dict[str, str] = {
    'Ş': 'Ș', 'ş': 'ș',
    'Ţ': 'Ț', 'ţ': 'ț',
}

def normalize_character(c: str) -> str:
    mapped = CEDILLA_MAPPING.get(c, c)
    return mapped.upper()

def normalize_text(text: str) -> str:
    return "".join(normalize_character(c) for c in text)
```

Comment: All input characters are converted to standard comma-below representations and cast to uppercase before any membership validation or cryptographic transformation occurs.

3.2.3 Whitespace Stripping and Strict Character Validation

Task 3.2 specifies: *"În mesaj mai întâi trebuie eliminate spațiile, apoi toate literele se vor transforma în majuscule. Valorile caracterelor textului sunt cuprinse între 'A' și 'Z', 'a' și 'z' și nu sunt permise alte valori. În cazul în care utilizatorul introduce alte valori – i se va sugera diapazonul corect al caracterelor."*

```
def validate_text(raw_text: str) -> Tuple[bool, Optional[str], str]:
    if not raw_text:
        return False, "Error: Text input cannot be empty.", ""

    # Task 3.2 requirement: spaces must be removed before uppercase transformation
    without_spaces = "".join(raw_text.split())
    if not without_spaces:
        return False, "Error: Text cannot contain only spaces.", ""

    normalized = normalize_text(without_spaces)
    invalid_chars = sorted(list(set(c for c in normalized if c not in CHAR_TO_INDEX
    )))

    if invalid_chars:
        return (
            False,
            f"Error: Invalid character(s) detected: {', '.join(repr(c) for c in
    invalid_chars)}.\n"
            f"Allowed characters: Only Romanian alphabet letters ('A'-'Z', 'a'-'z',
     including Ă, Â, Î, Ș, Ț).",
            "",
        )

```

```
    return True, None, normalized
```

Comment: Following the exact specification sequence, spaces are stripped before conversion. Any rejected character (digits, punctuation, symbols) is extracted and reported alongside the explicit allowed character interval.

3.2.4 Key Validation and Length Enforcement ($|K| \geq 7$)

The laboratory specification dictates that the key length cannot be less than 7 and must consist exclusively of Romanian alphabet letters:

```
def validate_key(raw_key: str) -> Tuple[bool, Optional[str], str]:
    if not raw_key:
        return False, "Error: Key cannot be empty.", ""

    without_spaces = "".join(raw_key.split())
    normalized = normalize_text(without_spaces)

    invalid_chars = sorted(list(set(c for c in normalized if c not in CHAR_TO_INDEX
    )))
    if invalid_chars:
        return (
            False,
            f"Error: Invalid character(s) in key: {', '.join(repr(c) for c in
    invalid_chars)}.\n"
            f"Allowed characters: Only Romanian alphabet letters ('A'-'Z', 'a'-'z',
     including Ă, Â, Î, Ș, Ț).",
            "",
        )

    if len(normalized) < 7:
        return (
            False,
            f"Error: Key length must be at least 7 characters (got {len(normalized)
    }: '{normalized}').",
            "",
        )

    return True, None, normalized
```

Comment: Spaces within the key string are removed, non-alphabet characters are rejected with diagnostics, and the strict $|K| \geq 7$ bound is enforced.

3.2.5 Vigenère Encryption and Decryption Core

The modular transformations map characters to numeric codes in $\mathbb{Z}_{31}$ and compute the shifted indices:

```
def encrypt_vigenere(plaintext: str, key: str) -> str:
    """Encrypts plaintext using Vigenere cipher: ci = (mi + ki) mod 31."""
    cipher_chars: List[str] = []
    key_len = len(key)

    for i, char in enumerate(plaintext):
        m_i = CHAR_TO_INDEX[char]
        k_i = CHAR_TO_INDEX[key[i % key_len]]
        c_i = (m_i + k_i) % ALPHABET_SIZE
```

```
        cipher_chars.append(INDEX_TO_CHAR[c_i])

    return "".join(cipher_chars)

def decrypt_vigenere(ciphertext: str, key: str) -> str:
    """Decrypts ciphertext using Vigenere cipher: mi = (ci - ki) mod 31."""
    plain_chars: List[str] = []
    key_len = len(key)

    for i, char in enumerate(ciphertext):
        c_i = CHAR_TO_INDEX[char]
        k_i = CHAR_TO_INDEX[key[i % key_len]]
        # Python's modulo operator handles negative numbers correctly: (-k) % 31 -> [0..30]
        m_i = (c_i - k_i) % ALPHABET_SIZE
        plain_chars.append(INDEX_TO_CHAR[m_i])

    return "".join(plain_chars)
```

Comment: Linear complexity $O(N)$ with respect to message length N. Floored modulo arithmetic in Python ensures that negative values $(c_i - k_i) < 0$ cleanly wrap into the valid interval $[0, 30]$.

3.2.6 Step-by-Step Mathematical Calculation Table

To provide full insight into intermediate values, `format_vigenere_table` produces the exact 6-row matrix format demonstrated in Section 3.1 of the laboratory guide:

```
def format_vigenere_table(text: str, key: str, is_encrypt: bool = True) -> str:
    """Formats calculation table matching the mathematical model shown in the lab guide."""
    key_len = len(key)
    repeated_key = [key[i % key_len] for i in range(len(text))]

    text_indices = [CHAR_TO_INDEX[c] for c in text]
    key_indices = [CHAR_TO_INDEX[k] for k in repeated_key]

    if is_encrypt:
        mod_indices = [(t + k) % ALPHABET_SIZE for t, k in zip(text_indices, key_indices)]
        result_chars = [INDEX_TO_CHAR[idx] for idx in mod_indices]
        lbl_text, lbl_key, lbl_op, lbl_res = "Plaintext M", "Key K", "M + K (mod 31)", "Ciphertext C"
    else:
        mod_indices = [(t - k) % ALPHABET_SIZE for t, k in zip(text_indices, key_indices)]
        result_chars = [INDEX_TO_CHAR[idx] for idx in mod_indices]
        lbl_text, lbl_key, lbl_op, lbl_res = "Ciphertext C", "Key K", "C - K (mod 31)", "Plaintext M"

    col_widths = [max(len(str(t)), len(str(k)), len(str(m)), len(str(rc)), 2)
                  for t, k, m, rc in zip(text, repeated_key, mod_indices, result_chars)]

    lines = [
        f"{lbl_text:<16}: " + " ".join(f"{c:>{w}}" for c, w in zip(text, col_widths)),
```

```
        f"{lbl_key:<16}: " + " ".join(f"{c:>{w}}" for c, w in zip(repeated_key, col_widths)),
        f"{lbl_text:<16}: " + " ".join(f"{idx:>{w}}" for idx, w in zip(text_indices, col_widths)),
        f"{lbl_key:<16}: " + " ".join(f"{idx:>{w}}" for idx, w in zip(key_indices, col_widths)),
        f"{lbl_op:<16}: " + " ".join(f"{idx:>{w}}" for idx, w in zip(mod_indices, col_widths)),
        f"{lbl_res:<16}: " + " ".join(f"{c:>{w}}" for c, w in zip(result_chars, col_widths)),
    ]
    return "\n".join(lines)
```

Comment: The function handles both encryption ($M+K$) and decryption ($C-K$), producing an aligned fixed-width textual matrix that matches the mathematical example in the lab manual.

4 Results and Demonstration

4.1 Task 3.2: Operational Verification and Step-by-Step Calculation

Consider encrypting the Romanian test phrase containing multiple words and diacritic letters:

"CRIPTOGRAFIA ESTE O ȘTIINȚĂ FUNDAMENTALĂ"

using the secret key $K =$ "SECURITATE" ($|K| = 10 \geq 7$).

1. **Preprocessing:** Spaces are stripped, cedillas converted, and characters cast to uppercase:

   "CRIPTOGRAFIAESTEOȘTIINȚĂFUNDAMENTALĂ"

   yielding $N = 36$ letters.

2. **Key Expansion:** The 10-character key "SECURITATE" is repeated across $N = 36$:

   "SECURITATESECURITATESECURITATESECURI"

3. **Step-by-Step Transformation (First 18 Characters):** Table 2 illustrates the 6-row mathematical calculation generated by the system.

Table 2: Task 3.2: Step-by-step Vigenère encryption calculation matrix (First 18 letters).

Plaintext M	C	R	I	P	T	O	G	R	A	F	I	A	E	S	T	E	O	Ș
Key K	S	E	C	U	R	I	T	A	T	E	S	E	C	U	R	I	T	A
Code M	4	20	10	18	23	17	8	20	0	7	10	0	6	21	23	6	17	22
Code K	21	6	4	25	20	10	23	0	23	6	21	6	4	25	20	10	23	0
$M+K \bmod 31$	25	26	14	12	12	27	0	20	23	13	0	6	10	15	12	16	9	22
Ciphertext C	U	V	L	J	J	W	A	R	T	K	A	E	I	M	J	N	H	Ș

The complete ciphertext produced for the full 36-letter plaintext is:

UVLJJWARTKAEIMJNHȘMNAȘXVWCGDTSWȘWUBÎ

4.2 Task 3.2: Round-Trip Decryption Verification

Applying the decryption procedure with the same key $K =$ `"SECURITATE"` to the generated ciphertext restores the plaintext message. Table 3 details the inverse arithmetic calculation.

Table 3: Task 3.2: Step-by-step Vigenère decryption calculation matrix (First 18 letters).

Ciphertext C	U	V	L	J	J	W	A	R	T	K	A	E	I	M	J	N	H	Ș
Key K	S	E	C	U	R	I	T	A	T	E	S	E	C	U	R	I	T	A
Code C	25	26	14	12	12	27	0	20	23	13	0	6	10	15	12	16	9	22
Code K	21	6	4	25	20	10	23	0	23	6	21	6	4	25	20	10	23	0
$C - K \bmod 31$	4	20	10	18	23	17	8	20	0	7	10	0	6	21	23	6	17	22
Plaintext M	C	R	I	P	T	O	G	R	A	F	I	A	E	S	T	E	O	Ș

The final decrypted text is identical to the original preprocessed message:

`CRIPTOGRAFIAESTEOȘTIINȚĂFUNDAMENTALĂ`

confirming complete algebraic round-trip fidelity.

4.3 Task 3.2: Error Handling and Input Validation

The implementation strictly validates user input and displays informative diagnostics before prompting again:

- **Short Key Enforcement** ($|K| < 7$)**:** Entering key `"PAROLA"` (length 6) is rejected with:

```
Error: Key length must be at least 7 characters (got 6: 'PAROLA').
```

- **Illegal Character Rejection:** Entering `"Salut 123!"` isolates characters `'!'`, `'1'`, `'2'`, `'3'` and indicates the permitted Romanian alphabet range:

```
Error: Invalid character(s) detected: '!', '1', '2', '3'.
Allowed characters: Only Romanian alphabet letters ('A'-'Z', 'a'-'z', including
     Ă, Â, Î, Ș, Ț).
```

- **Cedilla Normalization:** Entering legacy cedilla variants (e.g., `"ŞTIINŢĂ"`) automatically normalizes characters to comma-below letters (`"ȘTIINȚĂ"`), mapping to correct indices: $22, 23, 10, 10, 16, 24, 1$.
- **Whitespace Stripping:** Interspersed spaces in `"C R I P T O"` are automatically stripped before processing, fulfilling the explicit Task 3.2 requirement.

4.4 Interactive CLI Application

Executing `python3 main.py` greets the user with an interactive terminal interface designed consistently with the laboratory layout:

```
============================================================
 LABORATORY WORK NO. 3: VIGENÈRE CIPHER IMPLEMENTATION
 Course: Cryptography & Security | UTM FCIM FAF-243
 Author: Şoimu Ionuţ | Supervisor: Zaica Maia
============================================================
  1. Task 3.2: Vigenère Cipher over Romanian Alphabet
  2. Exit
============================================================
Enter choice (1-2): 1
```

```
--- TASK 3.2: VIGENÈRE CIPHER (ROMANIAN ALPHABET) ---
1. Encrypt plaintext
2. Decrypt ciphertext
3. Display Romanian alphabet table (31 letters, indices 0..30)
4. Return to main menu
--------------------------------------------------
```

5 Conclusions

Through this laboratory work, I implemented the polyalphabetic Vigenère cipher adapted to the 31-letter Romanian alphabet, fulfilling all requirements of Task 3.2.

- **Polyalphabetic Substitution Mechanics:** Unlike the single-key and permuted Caesar ciphers where each letter has a fixed image, the Vigenère cipher alternates across multiple Caesar shifts determined by the key string. This ensures that the same plaintext letter maps to different ciphertext symbols across the message, effectively flattening the frequency distribution.
- **Mathematical Model over $\mathbb{Z}_{31}$:** Restricting the domain to the 31-letter Romanian alphabet required establishing custom bijective mappings between letters and indices $0 \dots 30$, avoiding language-dependent ASCII offsets. Modular addition and subtraction accurately implement encryption ($c_i = (m_i + k'_i) \bmod 31$) and decryption ($m_i = (c_i - k'_i) \bmod 31$).
- **Input Preprocessing and Validation:** The system rigorously enforces the Task 3.2 requirements by removing whitespace before capitalization, validating character ranges, rejecting illegal characters with informative feedback, and normalizing legacy cedilla letters (Ş, Ţ) to standard comma-below characters (Ș, Ț).
- **Interactive Workflow and Step-by-Step Transparency:** The interactive CLI provides clear prompts, re-prompting on invalid input, and generates full 6-row mathematical tables displaying numeric indices and modular arithmetic results for every character.

In conclusion, this laboratory demonstrated the practical implementation of polyalphabetic substitution for non-English alphabets, confirming full compliance with the mathematical and functional requirements of Task 3.2.

6 Source Code Repository

The complete source code and report build files are organized in the project repository:

GitHub Repository Link:
https://github.com/Lucariowu/cslabs/tree/main/lab3